Relatório Meta 3: Classificação de Texto com Prompt Engineering

Disciplina: Processamento de Linguagem Natural (PLN)

Dataset: HateBR - Detecção de Discurso de Ódio em Português

Modelo Principal: Llama 3.2 (3B parâmetros) via Ollama

Alunos: Lorran Deiró e Pedro Sandoval

1. Introdução e Objetivos

1.1 Descrição da Meta

Nesta etapa do projeto, nosso objetivo foi desenvolver um sistema de classificação de texto utilizando técnicas de Prompt Engineering com modelos de linguagem open-source baseados em transformers. O trabalho precisava seguir alguns requisitos específicos: usar um modelo de linguagem com no máximo 8 bilhões de parâmetros, explorar múltiplas estratégias de prompting (Zero-Shot, Few-Shot e Chain-of-Thought), fazer uma análise comparativa com os resultados das metas anteriores, além de documentar adequadamente o modelo escolhido e realizar uma análise detalhada dos erros encontrados.

1.2 Fundamentação Teórica

As técnicas de Prompt Engineering que utilizamos são baseadas em literatura acadêmica consolidada. O Zero-Shot Prompting (Brown et al., 2020) permite que o modelo resolva a tarefa sem exemplos, apenas com instruções claras. O Few-Shot In-Context Learning (Brown et al., 2020; Liu et al., 2023) fornece exemplos anotados no próprio prompt para guiar o modelo. Já o Chain-of-Thought Prompting (Wei et al., 2022) solicita que o modelo explicite seu raciocínio passo a passo antes de chegar à resposta final.

2. Modelo Utilizado: Llama 3.2 (3B)

2.1 Especificações Técnicas

O modelo que escolhemos foi o Llama 3.2 com 3 bilhões de parâmetros, o que está bem dentro do limite exigido de 8B. Trata-se de uma arquitetura Transformer decoder-only, com janela de contexto de 2048 tokens, desenvolvido pela Meta AI, sob licença open-source. A execução foi feita localmente através do Ollama com aceleração por GPU.

2.2 Configuração do Modelo

Para garantir resultados reproduzíveis, configuramos o modelo de forma determinística:

```
llm = OllamaLLM(
```

```
    model="llama3.2:latest",
    temperature=0.0,       # Determinístico
    num_predict=10,        # Máximo de tokens
    num_ctx=512,           # Tamanho do contexto
    top_k=1,               # Greedy decoding
    top_p=0.1,             # Nucleus sampling restrito
    num_gpu=99,            # Todas as camadas na GPU
    num_thread=8           # Threads para preprocessing
)
```

2.3 Justificativa da Escolha

O Llama 3.2 foi selecionado por cumprir os requisitos da meta (menos de 8B parâmetros) e por permitir execução local via Ollama. Isso nos garantiu independência de conexão com a internet, velocidade de processamento significativamente maior que APIs online, e um ambiente controlado para reprodutibilidade dos experimentos.

3. Processo de Desenvolvimento

3.1 Jornada de Aprendizado

A descoberta do poder da GPU

- Quando começamos o projeto, estávamos executando tudo na CPU. O processamento era extremamente lento - estávamos olhando para horas e mais horas de espera para rodar os experimentos.
- Conversando com colegas e pesquisando na internet, descobrimos que configurar o Ollama para usar a GPU do computador faria uma diferença brutal no tempo de processamento.
- Após fazer essa mudança, o que antes levava horas passou a levar minutos.

Tentativas com outros modelos

- Antes de nos decidirmos pelo Llama 3.2 com 3B de parâmetros, também experimentamos outras opções.
- Uma delas foi o Llama 1B:
    - Modelo menor e mais rápido. Porém, ao testarmos, ficou claro que ele apresentava uma acurácia inferior nos resultados.
    - Por mais que fosse mais veloz, o desempenho não compensava sua utilização - preferimos esperar um pouco mais e ter resultados mais confiáveis.
- Também chegamos a considerar usar a API do Grok (xAI) como alternativa:
    - No entanto, rapidamente percebemos que não seria viável: a API tinha um limite de apenas 30 requisições por hora, o que tornaria impossível processar nossos 1.750 textos de teste em tempo hábil - seriam necessárias cerca de 58 horas só para rodar um experimento.
    - Além disso, havia a dependência de conexão com a internet e a latência variável das requisições.
    - A execução local via Ollama se mostrou muito mais prática.

3.2 Evolução dos Prompts

Uma das partes mais interessantes do projeto foi aprender a construir bons prompts. Desde o início, já incluímos os critérios de classificação nos prompts - isso não foi uma descoberta tardia, mas sim algo que fomos refinando ao longo de todo o desenvolvimento. O que mudou significativamente foi a forma como estruturamos esses critérios e a quantidade de informação que tentávamos passar para o modelo.

O problema da complexidade excessiva

Nossa intuição inicial foi a de que "quanto mais informação, melhor o resultado". Criamos prompts extremamente detalhadas:

- Explicações completas do projeto, contexto acadêmico extenso, definições elaboradas de discurso de ódio, múltiplos exemplos de critérios.
- No caso do Few-Shot, chegamos a tentar usar todos os exemplos do dataset de treino como contexto - o que claramente ultrapassa a janela de contexto do modelo.

Resultado: O modelo simplesmente não conseguia processar tanta informação. Recebíamos respostas truncadas, incoerentes, ou o modelo simplesmente recusava classificar. Era comum ver mensagens como "Não posso fornecer uma classificação..." ou textos cortados no meio.

Simplificação gradual

A solução foi um processo iterativo de simplificação. Fomos removendo elementos aos poucos: primeiro as explicações do projeto, depois reduzimos drasticamente o número de exemplos no Few-Shot (de centenas para apenas 16 exemplos balanceados), simplificamos os critérios para categorias binárias mais claras, e finalmente adicionamos uma constraint explícita no formato de saída. Aprendemos que pedir "RESPONDA APENAS COM O NÚMERO: 0 ou 1" era fundamental para evitar respostas longas ou ambíguas.

Versão final dos prompts

Zero-Shot:

```
Você é um classificador de texto para pesquisa acadêmica (dataset HateBR).

Classifique este texto: "{texto}"
Categoria 1 = linguagem ofensiva (palavrões, insultos, agressividade)
Categoria 0 = linguagem não ofensiva (neutra, respeitosa)
RESPONDA APENAS COM O NÚMERO: 0 ou 1
```

Few-Shot:

```
Você é um classificador calibrado com 16 exemplos de treinamento

(8 ofensivos, 8 não ofensivos) do dataset HateBR. [+ mesma estrutura]
```

Chain-of-Thought:

```
Antes de classificar, análise:

1. Há palavrões, xingamentos ou insultos diretos?
2. O tom é agressivo, hostil ou desrespeitoso?
3. Há ataques pessoais, ameaças ou incitação ao ódio?
4. A linguagem é ofensiva ou depreciativa?
```

Lições aprendidas

1. Menos é mais quando se trata de prompts;
2. Constraints explícitos no formato de saída reduzem drasticamente erros de parsing; no Few-Shot, a qualidade e o balanceamento dos exemplos superam a quantidade;
3. Mencionar "pesquisa acadêmica" no prompt ajuda a reduzir recusas do modelo por questões de conteúdo sensível.

4. Resultados e Análise de Desempenho

4.1 Configuração Experimental

Dividimos o dataset em 5.250 exemplos para treino (usado para selecionar os exemplos do Few-Shot) e 1.750 para teste. A distribuição estava balanceada, com aproximadamente 50% de cada classe. Todos os experimentos foram executados localmente com aceleração por GPU.

4.2 Resultados das Três Estratégias

Estratégia	Acurácia	Precisão	Recall	F1-Score
Zero-Shot	76.57%	81.63%	68.57%	74.53%
Few-Shot	71.71%	69.00%	78.86%	73.60%
Chain-of-Thought	72.97%	66.16%	94.06%	**77.68%**

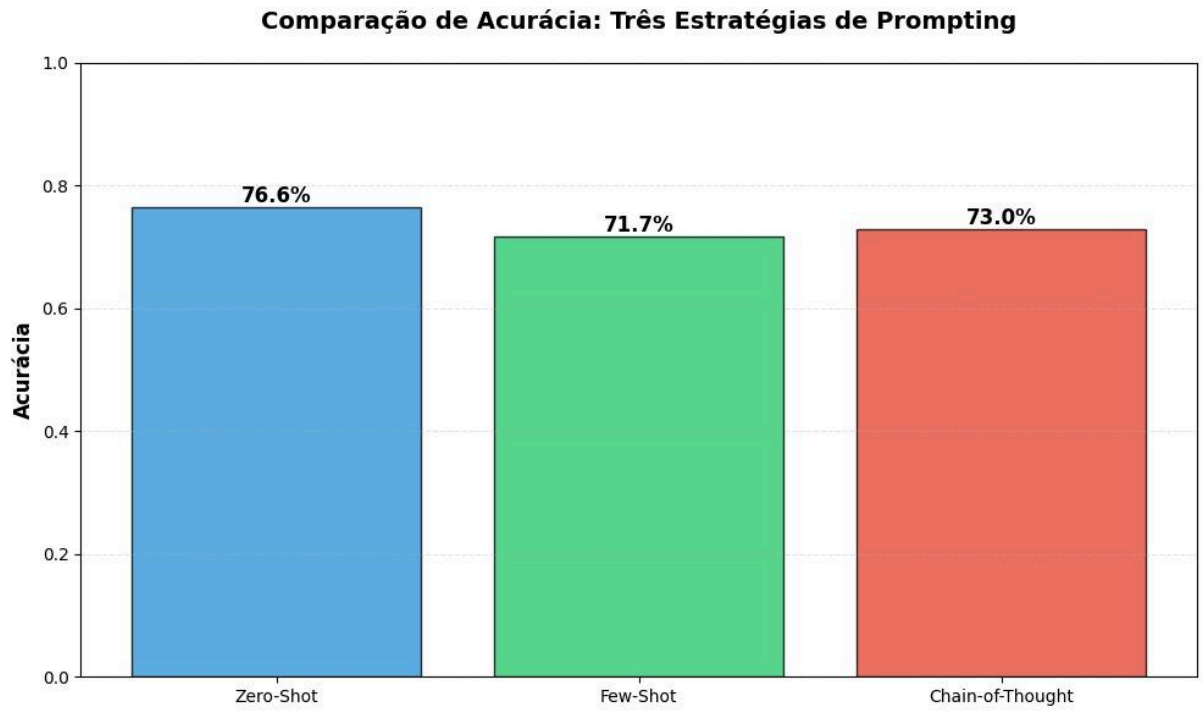
4.3 Análise dos Resultados

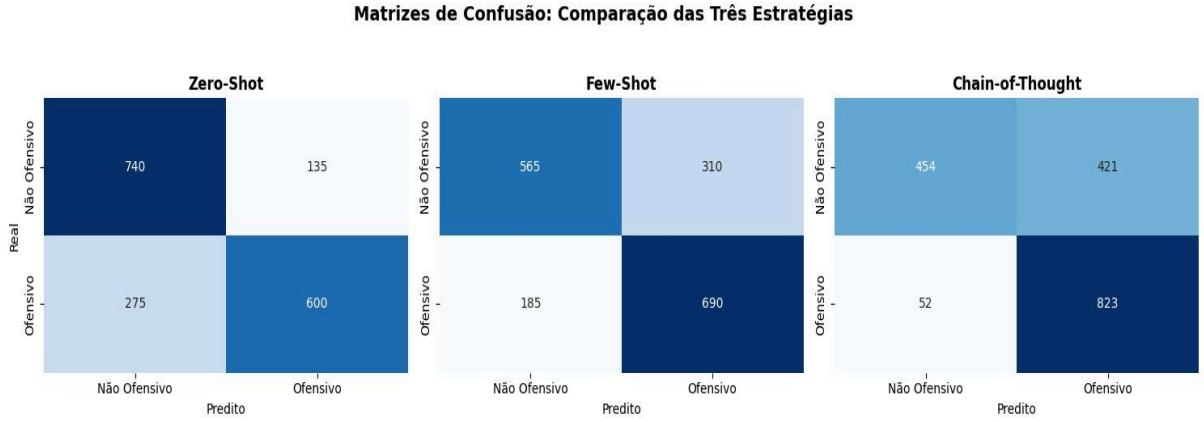
Cada estratégia apresentou características bem distintas:

Zero-Shot: Obteve a maior acurácia (76.57%), mas com o menor recall (68.57%). O modelo mostrou uma tendência conservadora, classificando menos textos como ofensivos. Quando ele classifica algo como ofensivo, porém, acerta com mais frequência (precisão de 81.63%).

Few-Shot: Apresentou desempenho intermediário em todas as métricas. Curiosamente, a calibração com exemplos não trouxe ganho significativo neste caso específico - talvez porque os 16 exemplos que selecionamos não foram representativos o suficiente da diversidade do dataset.

Chain-of-Thought: Alcançou o melhor F1-Score (77.68%), que é a métrica mais adequada para dados balanceados. O recall foi excepcional (94.06%), detectando quase todos os textos ofensivos. Em contrapartida, teve menor precisão (66.16%), gerando mais falsos positivos. O tempo de processamento também foi maior (0.88s vs 0.57s) devido ao raciocínio explícito.





4.4 Comparação com Meta 2

Como Desejado no enunciado, aplicamos uma comparação das três abordagens(3 metas implementadas), as quais foram completamente diferentes para o mesmo problema: o classificador baseado em regras da Meta 1, os modelos de machine learning da Meta 2, e agora os LLMs com prompt engineering da Meta 3. Cada abordagem tem suas características e trade-offs.

Modelo	Acurácia	F1-Score	Tipo
Regras (Meta 1)	**88.00%**	**88.00%**	Baseado em Regras
Naive Bayes (Meta 2)	84.86%	85.04%	ML Tradicional
SVM Linear (Meta 2)	84.11%	83.89%	ML Tradicional
Regressão Logística (Meta 2)	83.26%	83.17%	ML Tradicional
CoT - Llama 3.2 (Meta 3)	72.97%	77.68%	LLM
Zero-Shot - Llama 3.2 (Meta 3)	76.57%	74.53%	LLM
Few-shot - Llama 3.2 (Meta 3)	71.71%	73.60%	LLM

Reflexões sobre os resultados

- **Classificador baseado em regras da Meta 1:**
  - Obteve o melhor desempenho geral (F1 de 88%), superando inclusive os modelos de machine learning da Meta 2.
  - Isso nos fez refletir sobre a natureza do dataset HateBR: aparentemente, o discurso de ódio nesse corpus é predominantemente explícito, com palavras-chave bem definidas que um sistema de regras consegue capturar eficientemente.
- **Modelos de ML da Meta 2:**
  - Ficaram logo atrás, com o Naive Bayes atingindo a F1 de 85.04%.
  - A diferença para o sistema de regras não é tão grande, mas mostra que às vezes uma abordagem mais simples e interpretável pode ser tão boa quanto - ou até melhor que - algoritmos mais sofisticados.
- **LLM da Meta 3:**
  - Mesmo com nossa melhor estratégia (Chain-of-Thought), ficou cerca de 10 pontos percentuais abaixo do sistema de regras.
  - Faz sentido quando consideramos que o LLM não foi treinado especificamente para o português brasileiro nem para o domínio de discurso de ódio em redes sociais.
  - Ele está generalizando a partir de seu conhecimento prévio, sem acesso ao vocabulário específico que construímos na Meta 1.
- **Importante destacar:**
  - O LLM tem uma vantagem única: ele conseguiu esse resultado sem nenhum treino, apenas com instruções em linguagem natural.]
  - Para cenários onde não temos dados anotados ou precisamos de uma solução rápida, essa flexibilidade é extremamente valiosa.

5. Análise de Erros

5.1 Padrões de Erro por Estratégia

Zero-Shot - Alta taxa de Falsos Negativos: O principal problema foi não detectar textos ofensivos que usavam sarcasmo ou ironia, linguagem ofensiva implícita, e gírias ou expressões regionais brasileiras que o modelo não reconheceu como ofensivas.

Few-Shot - Padrão balanceado de erros: Não identificamos um padrão claro de erros específico para esta estratégia. Os erros se distribuíram de forma relativamente equilibrada entre falsos positivos e falsos negativos, sem uma tendência marcante. Isso sugere que os 16 exemplos de calibração ajudaram a manter um equilíbrio, mesmo que não tenham melhorado significativamente o desempenho geral.

Chain-of-Thought - Alta taxa de Falsos Positivos: O modelo se mostrou "sensível demais": classificou como ofensivos textos que apenas mencionam palavras potencialmente ofensivas em contexto neutro, citações ou reportagens sobre discurso de ódio, e críticas legítimas que interpretou como ataques.

5.2 Recusas de Classificação

Durante a execução, o modelo ocasionalmente recusou classificar alguns textos, respondendo com mensagens como "Desculpa, mas não posso classificar..." ou "Não posso classificar linguagem ofensiva". Implementamos um sistema de fallback com múltiplos padrões regex para tratar esses casos. Textos não classificados foram tratados com um default conservador (não ofensivo). Felizmente, a taxa de recusa ficou abaixo de 1% do total.

5.3 Exemplos Representativos

Falsos Negativos típicos: Textos com xingamentos regionais que o modelo não reconheceu, ofensas veladas ou indiretas, e sarcasmo que inverte o significado literal das palavras.

Falsos Positivos típicos: Textos que mencionam palavras ofensivas de forma descritiva ("ele disse X"), notícias ou reportagens sobre hate speech, e críticas duras mas legítimas a figuras públicas.

6. Métricas de Avaliação: Justificativa

6.1 Por que escolhemos o F1-Score

Escolhemos o F1-Score como métrica principal por três razões: ele equilibra a capacidade de detectar ofensas (recall) com a confiabilidade das detecções (precisão); é adequado para o dataset HateBR que possui distribuição equilibrada entre as classes; e penaliza modelos que maximizam apenas uma dimensão em detrimento da outra.

6.2 Importância de cada métrica no contexto

Precisão: Responda "quantos dos textos marcados como ofensivos realmente são?". Importante para evitar censura indevida de conteúdo legítimo.

Recall: Responde "quantos textos ofensivos conseguimos detectar?". Importante para proteção efetiva de usuários contra conteúdo nocivo.

Acurácia: Oferece uma visão geral, mas pode ser enganosa em datasets desbalanceados (não é nosso caso, mas é bom ter em mente).

F1-Score: Síntese equilibrada das duas métricas anteriores, nossa métrica de decisão principal.

7. Conclusões

7.1 O que aprendemos

- Este projeto, ao longo das três metas, nos proporcionou uma visão abrangente de diferentes abordagens para classificação de texto.
- **Aprendizados principais:**
    - O classificador baseado em regras da Meta 1 surpreendeu ao obter o melhor desempenho geral (F1 de 88%), mostrando que para datasets

com discurso de ódio explícito, regras bem construídas são muito eficazes.
    - Os modelos ML da Meta 2 ofereceram bom desempenho com menos esforço manual, mas sem superar as regras neste caso específico.
    - E o LLM da Meta 3, mesmo com desempenho inferior, mostrou uma flexibilidade impressionante ao resolver a tarefa sem nenhum treino específico.
- Aprendemos também que Chain-of-Thought foi a melhor estratégia de prompting para nosso problema, que simplicidade é essencial em prompts (nossa intuição de "mais é melhor" estava errada), e que a execução local via Ollama é uma opção viável e muito mais rápida que APIs online.

7.2 Trade-offs identificados

Ao longo das três metas, pudemos observar claramente os trade-offs de cada abordagem:

Aspecto	Regras (Meta 1)	ML (Meta 2)	LLMs (Meta 3)
Desempenho	**Melhor (88% F1)**	Bom (~85% F1)	Inferior (~77% F1)
Esforço inicial	Alto (criar léxico)	Médio (treinar)	**Baixo (prompt)**
Interpretabilidade	**Alta (regras explícitas)**	Baixa	Alta com CoT
Manutenção	Trabalhosa (atualizar léxico)	Retreinar modelo	**Ajustar prompt**
Generalização	Baixa (domínio fixo)	Média	**Alta (adaptável)**

7.3 Aprendizados práticos

- Com base em nossa experiência nas três metas, foi idealizado que:
    - Quando há conhecimento do domínio e tempo para construir um léxico, o sistema de regras pode ser surpreendentemente eficaz e totalmente interpretável.
    - Para cenários com dados anotados mas sem expertise linguística, os modelos de ML tradicionais oferecem um bom equilíbrio entre desempenho e facilidade de implementação.
    - Para prototipagem rápida, novos domínios sem dados anotados, ou quando a interpretabilidade do raciocínio é importante, LLMs com Chain-of-Thought são uma excelente opção.
    - E para sistemas em produção que precisam lidar com linguagem em constante evolução, uma abordagem híbrida combinando regras (alta precisão) com LLMs (generalização) pode ser o caminho ideal.

— Fim do Relatório —